

1. Design decisions

1.1 The framework

The framework of this design is a one-way dependency chain design; each module is hierarchically arranged based on its functionalities. The fundamental layer `stats.py` is used to support `preprocessing.py`, and the data preprocessed is passed to `tree.py` and `ensemble.py`. The pipeline concatenates the mentioned above together by throwing a unified training API interface like `partial.fit()` or `fit()`. Finally, `stream.py` controls the entire incremental learning process. `metrics.py` and `visualise.py` are separated from the training chain and are used to log or visualise relevant metrics during training. This design not only avoids high coupling but also makes sure it can be tested separately and modularly.

1.2 stats.py

`StreamStats` provides the preprocessing layer with mean, variance, quantiles and histogram, and all are calculated from incremental chunks. For values of mean and variance, this module uses the Welford algorithm to calculate them in parallel and incrementally instead of storing all dataset. Also, it prevents **catastrophic cancellation**. The quantiles and histogram calculations are controlled by the configuration `window_size`. When not given, only the distribution of the latest chunk is reflected; after being given, `deque(maxlen=window_size)` is used to buffer the fixed number of the latest chunks.

1.3 preprocessing.py

`Imputer` uses two different ways to fill in empty values based on given strategies. If the strategy is set as **mean**, `Imputer` fills the numeric data with the streaming mean. If the strategy is set as **mode**, `Imputer` fills the categories data with the most frequent category in the relevant feature. The mode is determined by the first chunk encountered in order to reduce memory usage.

`StandScaler` scale the new chunk by the values of **mean** and **variance** provided by `StreamStats`. In order to prevent repetitive computation, only the new chunk should be transformed by the new streaming values of **mean** and **variance**.

After the first `partial_fit()`, `OneHotEncoder` locks the category collection and the dimension of category data is fixed after that. Later unseen categories will be mapped to the **unknown** column.

1.4 tree.py

DecisionTreeClassifier implements the decision tree limited by depth, supporting two criteria: **Gini** and **Entropy**. All the functions are implemented using vectorised numpy. The configuration max_feature manages to choose the features of the chunk randomly to support **Random Forest**.

Also, in order to fit in streaming scenarios, partial_fit() provides window_size configuration. When set, the tree will be built on the buffering chunk data from zero repeatedly to automatically forget the old expired data rather than training on the incrementally accumulated data. And it is used to prevent the overfitting situation.

1.5 ensemble.py

EnsembleClassifier implements two ensemble strategies: **Bagging** and **Random Forest**, and manages n_estimators separate decision trees. When partial_fit() is called, the model **bootstrap samples** the chunk and fits the trees separately to make sure the robustness compared to a single tree. When max_feature is set when using the ensemble method, Random Forest chooses partly features randomly.

At the prediction period, the ensemble uses a **majority voting strategy**: it collects predictions of all trees and chooses the most frequent output as the final output. It is implemented by Counter.most_common(1)

1.6 stream.py

StreamTrainer is in charge of all training processes and logging memory usage and training time. fit_chunk() manipulates tracking training time and memory usage(using tracemalloc and time). score_chunk() manages to predict the outputs and log all metrics.

2. Testing & edge cases

This testing includes **six** modules, including **50** unit tests, using **unittest** Python library. And it uses coverage to demonstrate the coverage rate, it shows below that the total coverage rate is **91%**.

For functional testing, this test validates the correctness and consistent dimensions of outputs based on different modules. For instance, use StreamingStats to update the same chunk twice, assert the mean is 1.0, and the variance is 0.0. Also, it validates the correction of the Welford.

For rolling window testing, it validates the correct functionality when window_size is set. For example, when updating two different chunks four times, and the window_size is set as two, it shows that the quantile only reflects the distribution of the latest two chunks and confirms the effect of the deque's automatic elimination.

For edge cases testing, it covers below scenarios. Inputting an empty dataset raises ValueError; Incorrect calling order raises RuntimeError; The shape of y_true is not consistent with the shape of y_pred raising ValueError; Incorrect configuration raises ValueError(like criterion=Test);

```

. ok
test_one_hot_encoder (tests.test_preprocessing.TestPreprocess.test_one_hot_encoder) ... ok
test_scaler_transform_before_fit_raise (tests.test_preprocessing.TestPreprocess.test_scaler_transform_before_fit_raise) ... ok
test_standard_scaler (tests.test_preprocessing.TestPreprocess.test_standard_scaler) ... ok
test_transform_before_fit_raise (tests.test_preprocessing.TestPreprocess.test_transform_before_fit_raise) ... ok
test_get_histograms (tests.test_stats.TestStreamingStats.test_get_histograms) ... ok
test_get_quantiles (tests.test_stats.TestStreamingStats.test_get_quantiles) ... ok
test_mean_variance (tests.test_stats.TestStreamingStats.test_mean_variance) ... ok
test_rolling_histogram (tests.test_stats.TestStreamingStats.test_rolling_histogram) ... ok
test_rolling_quantiles (tests.test_stats.TestStreamingStats.test_rolling_quantiles) ... ok
test_wrong_order_hist_raise (tests.test_stats.TestStreamingStats.test_wrong_order_hist_raise) ... ok
test_wrong_order_mean_raise (tests.test_stats.TestStreamingStats.test_wrong_order_mean_raise) ... ok
test_wrong_quantiles_raise (tests.test_stats.TestStreamingStats.test_wrong_quantiles_raise) ... ok
test_fit_train (tests.test_tree.TestDecisionTreeClassifier.test_fit_train) ... ok
test_invalid_input_raise (tests.test_tree.TestDecisionTreeClassifier.test_invalid_input_raise) ... ok
test_predict_no_fit (tests.test_tree.TestDecisionTreeClassifier.test_predict_no_fit) ... ok
test_tree_depth_predictive (tests.test_tree.TestDecisionTreeClassifier.test_tree_depth_predictive) ... ok
... ok

-----
Ran 50 tests in 2.028s

OK
PROJECT_ROOT_PATH is set, using d:\Study\2026_Semester1\Programming_for_AI\Program\Assignment2\ProgrammingForAI-Assignment-2.2 as cwd for execution payload
There is no specific file
There is no specific file
Finished running tests!

-----

Ran 50 tests in 2.686s

OK

D:\Study\2026_Semester1\Programming_for_AI\Program\Assignment2\ProgrammingForAI-Assignment-2.2>python -m coverage report
Name                               Stmtms  Miss  Cover
-----
numcompute_stream\__init__.py       0      0  100%
numcompute_stream\ensemble.py       17      0  100%
numcompute_stream\io.py             35      6   83%
numcompute_stream\metrics.py       148     43   71%
numcompute_stream\pipeline.py       103      6   94%
numcompute_stream\preprocessing.py  112      8   93%
numcompute_stream\stats.py          70      2   97%
numcompute_stream\tree.py           80     11   86%
tests\test_ensemble.py              28      1   96%
tests\test_io.py                    24      1   96%
tests\test_metrics.py               89      1   99%
tests\test_pipeline.py              60      1   98%
tests\test_preprocessing.py         91      1   99%
tests\test_stats.py                 64      1   98%
tests\test_tree.py                  36      1   97%
-----
TOTAL                               957     83   91%

```

3. benchmark results

3.1 Origin and more vectorised

The benchmark compares the memory and training time between the original implementation and the more vectorised version based on the best_split function of the decision tree. It is evaluated on different sizes of data(**from 100 to 10000**) when using two different versions. And the results show as follows.

Size	Version	Time(secnonds)	Memory(MB)
#####			
100	Origin	0.0875	0.0059
100	Vectorised	0.074	0.0103
#####			
500	Origin	0.0937	0.0119
500	Vectorised	0.0724	0.021
#####			
1000	Origin	0.083	0.02
1000	Vectorised	0.0728	0.0343
#####			
5000	Origin	0.0993	0.0848
5000	Vectorised	0.0763	0.1402
#####			
10000	Origin	0.1026	0.1659
10000	Vectorised	0.0749	0.2737
#####			

This paragraph demonstrates that the training time of the more vectorised version is always lower than the original version, and the difference keeps continuously increasing as the size expands.

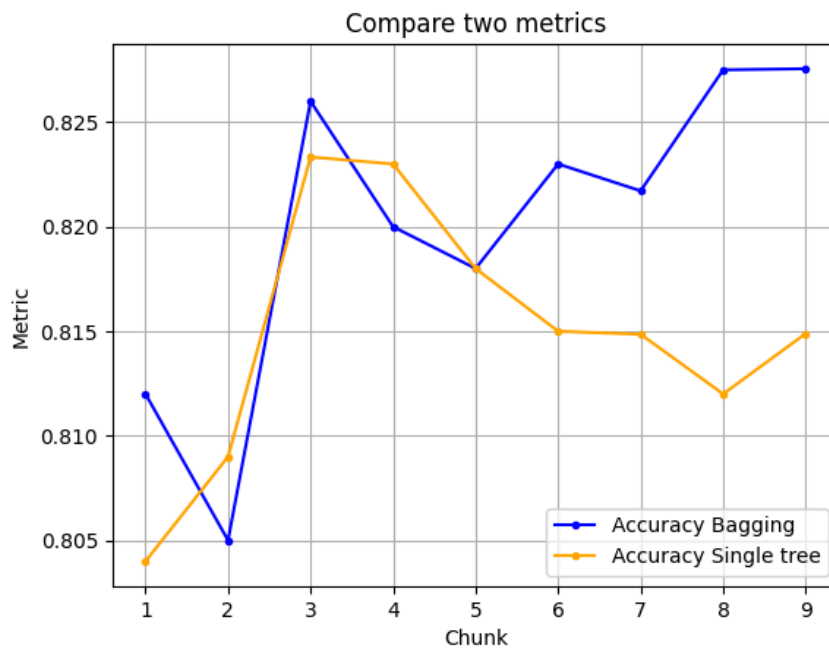
Considering the memory usage, the vectorised version is about 1.6 times higher than the original version because it requires additional memory spaces. But in the streaming scenarios, the size of every chunk is limited, and this trade-off is acceptable.

3.2 Models(Bagging VS Single Tree)

The benchmark trains one ensemble bagging method(`n_estimator = 5`) and one single decision tree on the Adult dataset. The dataset is set as follows: `chunk_size = 500`, the chunk number is 10. Both use a '**predict fisrt and then train**' strategy to simulate the real scenarios.

As the graphic indicates, the single tree has the basic learning ability on this data set. The ensemble method has a continuously higher accuracy rate after the fifth chunk compared to the single tree. The curve becomes smoother, and the fluctuation decreases overall as the amount of data increases.

In conclusion, the ensemble method is more advanced and robust compared to a single tree.



4. Reflection

The design of the framework has almost met the expected design requirements, and all modules are connected by the streaming interfaces. **50** unit tests demonstrate the **91%** code coverage rate. But there are still some misunderstandings in the design and unit tests.

`test_metrics.py` only covers **71%** code of `metrics.py`. Maybe I should add some more relevant testing.

`StreamMetrics.get_quantiles()` only demonstrates the quantile of the current chunk when `window_size` is not set; we should inform users before they use it.

Also, when users use `ensemble.py` to train a Random Forest, we should set the `n_estimators` to a relatively higher number (like 100), otherwise, the accuracy would be lower than expectations, as is shown in the demo.